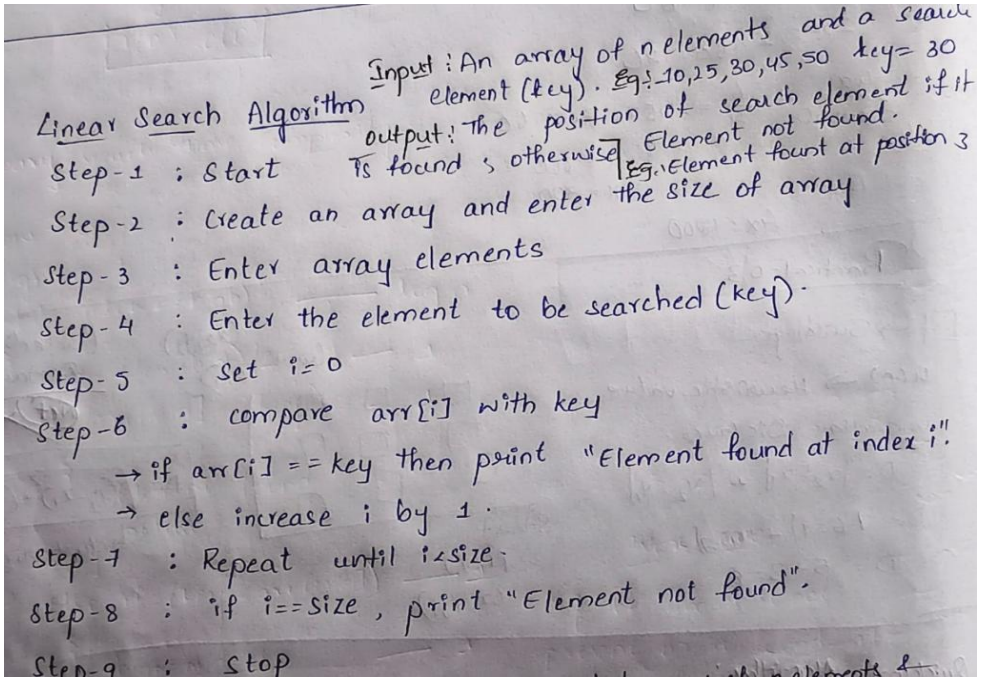
Practical - 2

Aim: To Implementation and Time analysis of linear and binary search algorithm.

1) Linear Search

Algorithm:



Program:

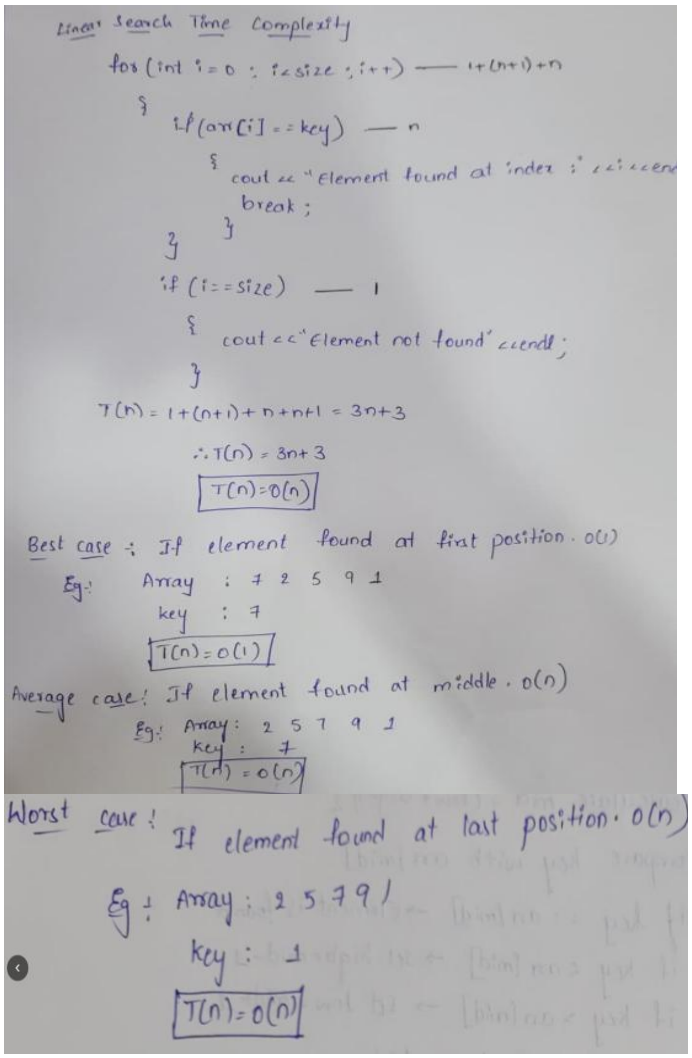
```
#include <iostream>
using namespace std;
class LinearSearch
{
public:
   int arr[100], n, key;

   void arrayInput()
   {
      cout << "Enter number of elements: ";
      cin >> n;

      cout << "Enter the elements: ";
      for (int i = 0; i < n; i++)
      {
         cin >> arr[i];
      }

      cout << "Enter the element to search: ";
      cin >> key;
```

```
void printArray()
    {
      for (int i = 0; i < n; i++)
      {
        cout << arr[i] << " ";
      }
      cout << endl;
    }

    void search()
    {
      int i;

      for (i = 0; i < n; i++)
      {
        if (arr[i] == key)
        {
          cout << "Element found at index " << i << endl;
          break;
        }
      }

      if (i == n)
      {
        cout << "Element not found." << endl;
      }
    }
};

int main()
{
  LinearSearch ls;

  ls.arrayInput();

  cout << "\nArray: ";
  ls.printArray();

  ls.search();

  return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ linear.cpp -o linear.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\linear.exe
Enter number of elements: 5
Enter the elements: 3 5 7 8 2
Enter the element to search: 7

Array: 3 5 7 8 2
Element found at index 2
```

Time Complexity:



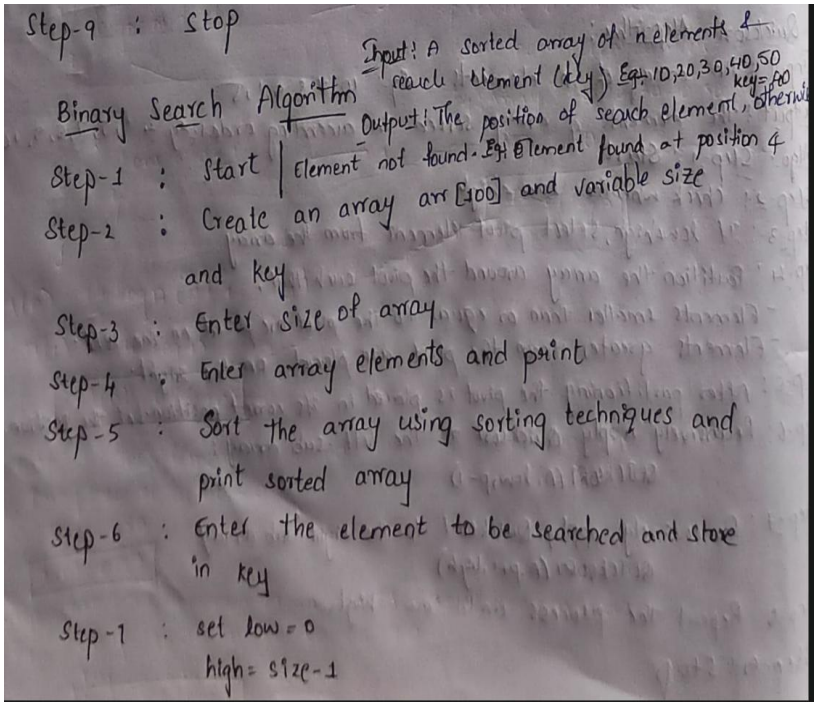
Linear search Conclusion :

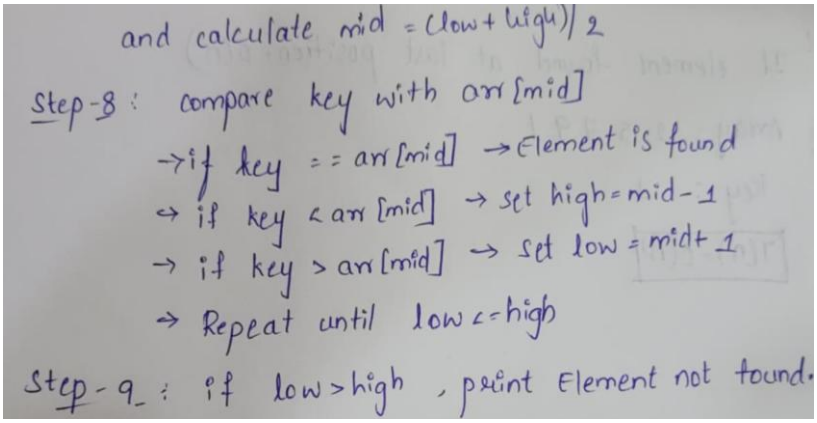
Thus, the Linear Search algorithm was successfully implemented and studied. It searches for an element by checking each element of the array one by one until the required element is found. It is simple and easy to implement, with a best-case time complexity of O(1) and average and worst-case time complexity of O(n).

Name : Vemalatha Bhimireddy
Enrollment Number:92460118162

2) Binary Search:

Algorithm:





Program :

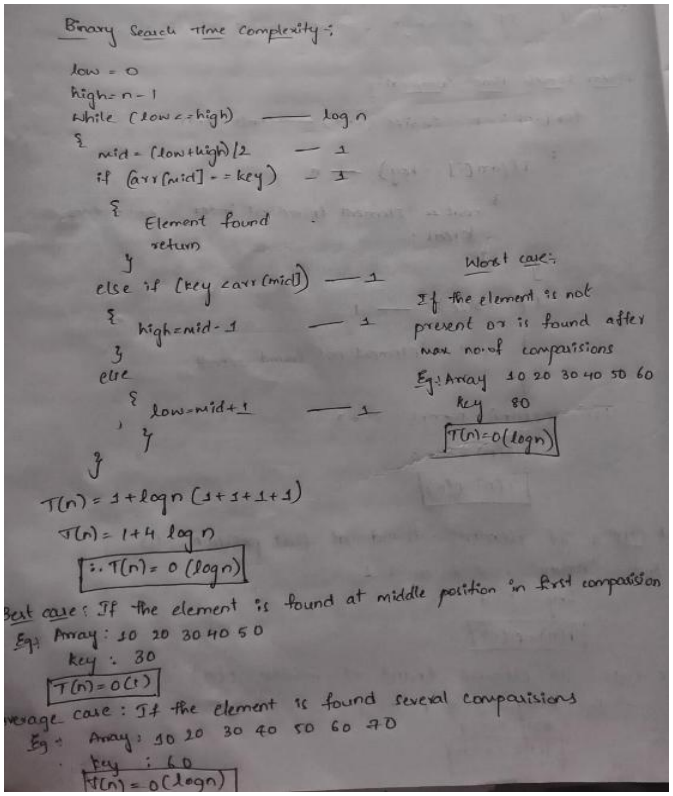
```
#include <iostream>
using namespace std;

class BinarySearch
{
public:
   int arr[100], n, key;

   void arrayInput()
   {
      cout << "Enter number of elements: ";
      cin >> n;

      cout << "Enter the sorted elements: ";
      for (int i = 0; i < n; i++)
      {
         cin >> arr[i];
      }

      cout << "Enter the element to search: ";
      cin >> key;
   }

   void printArray()
   {
      for (int i = 0; i < n; i++)
      {
         cout << arr[i] << " ";
      }
      cout << endl;
   }

   void search()
   {
      int low = 0;
      int high = n - 1;

      while (low <= high)
      {
         int mid = (low + high) / 2;

         if (arr[mid] == key)
         {
            cout << "Element found at index " << mid << endl;
            return;
         }
```

```
            else if (key < arr[mid])
            {
                high = mid - 1;
            }
            else
            {
                low = mid + 1;
            }
        }

        cout << "Element not found." << endl;
    }
};

int main()
{
    BinarySearch bs;

    bs.arrayInput();

    cout << "\nArray: ";
    bs.printArray();

    bs.search();

    return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ binary.cpp -o binary.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\binary.exe
Enter number of elements: 5
Enter the sorted elements: 10 45 67 86 78
Enter the element to search: 45

Array: 10 45 67 86 78
Element found at index 1
```

Time complexity:



Binary Search Conclusion :

Thus, the Binary Search algorithm was successfully implemented and studied. It searches for an element by repeatedly dividing the sorted array into two halves and checking the middle element. It is faster than Linear Search for large datasets, with a best-case time complexity of O(1) and average and worst-case time complexity of O(log n).

Conclusion :

Thus, we successfully implemented Linear Search and Binary Search and understood their working and time complexities. Linear Search checks elements one by one, while Binary Search repeatedly divides the sorted array into halves. We observed that Binary Search is more efficient for large sorted arrays, with a time complexity of O(log n) compared to O(n) for Linear Search.

Name : Vemalatha Bhimireddy
Enrollment Number:92460118162